

Chapter 10

Disease Classification and Detection in Plants

Introduction

In agriculture, plant diseases have always been a major problem, as they cause crop quality to decrease, resulting in yield. Plant disease has a range of effects from minor symptoms to serious damages in whole crop areas, which lead to high financial costs and significantly impact the agricultural economy, particularly in developing countries dependent on a single crop or on a few crops.

Various approaches have been developed to detect disease in order to avoid significant losses. Causal agents have been precisely defined by methods developed in molecular biology and immunology. However, for many farmers these techniques are inaccessible and require detailed knowledge of the field or substantial time and energy to be used. According to the United Nations Food and Agriculture Organization, most farms in the world are small and family-owned in developed countries. Such families provide food for a significant proportion of the population of the planet. However, hunger and food insecurity are not rare and there is restricted access to markets and services. For the above reasons, a great deal of work has been undertaken in order to establish methods that are sufficiently reliable and available to most farmers. Where Plant disease is an essential factor contributing to a major decline in plant production quality and quantity. Plant disease identification and classification are a significant role in improving plant production and economic development.

As it come to notice that how plant disease has been a major concern in the agriculture and many technologies are been used to detect it with an ease and artificial intelligence is one of them.

As an example, for plant disease detection and classification, here by using Artificial Intelligence's deep learning a model will be made and trained based on dataset from Kaggle which is plant village. It can be useful for identifying real-time plant disease In the Project we will make a CNN model which will be used for classification and detection of disease in potato based on the data we train it with.

Installing Packages

To develop a model or project on plant disease detection we will be requiring several libraries of Artificial Intelligence and python. Here we have discussed step by step their Installation and description for better understanding.

To install all the packages of libraries we will use PIP in windows. The libraries which are used to develop this project are:

- Keras: Command to install on windows: `pip install keras`
- NumPy: Command to install on windows: `pip install keras`
- Math: Command to install on windows: `pip install math`
- Matplotlib: Command to install on windows: `pip install matplotlib`
- TensorFlow: Command to install on windows: `pip install tensorflow`

Dataset

We are here taking a Dataset name Plant village from Kaggle which has 15 categories based on different crop diseases. We have selected to detect disease in potato plant for which we have 3 categories like Early Blight, Late Blight and healthy potato. For each these classes we will be having images as an input to train the model for prediction.

Dataset can be download from here. <https://www.kaggle.com/emmarex/plantdisease>

Implementation Steps

In the very first step of disease detection we will first import all the libraries required for the development for project. In this project development google research Collaboratory has been used and it can be accessed by this link <http://colab.research.google.com/> and you can also use the Anaconda for running this project but please specify all the path related to dataset and testing images into the project.

Step 1: First we will import the basic libraries which are required to develop this disease detection project are

```
import numpy as np
import math
import keras
import matplotlib.pyplot as plt
from tensorflow.keras.layers import *
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from tensorflow.keras import models
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.optimizers import *
from tensorflow.keras.callbacks import LearningRateScheduler
```

Output: Using TensorFlow backend.

These are all the libraries which will be required and processed through the project for various purposes.

Step 2: There are two methods by which we can import the data for the project
First method:

```
!pip install kaggle
!mkdir .kaggle
```

```
import json
token = {"username":"","key":""}
with open ('\content/.kaggle/kaggle.json', 'w') as file:
    json.dump (token, file)

!chmod 600 /content/.kaggle/kaggle.json
!cp /content/.kaggle/kaggle.json /root/.kaggle/kaggle.json
!kaggle datasets download -d xabdallahali/plantvillage-dataset/
color -p /content
```

Output:

```
Downloading plantvillage-dataset.zip to /content
99% 2.02G/2.04G [00:44<00:00, 0.6MB/s]
100% 2.04G/2.04G [00:44<00:00, 1.6MB/s]
```

Here we directly download the dataset from the Kaggle you can use it as it in your file or you can use the second method.

Second Method: If you are using the google research Collaboratory then you can download the dataset from this link <https://www.kaggle.com/emmarex/plantdisease> and upload it in your google drive “Colab Notebooks” folder then you can link it by using this code

```
from google.colab import drive
drive.mount ('/gdrive')
```

Output:

Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=94.....y

Enter your authorization code:

.....

Mounted at /gdrive

This is the code for mounting your drive with google research Collaboratory which will ask you authentication which you can do it with your Gmail account by just following the simple steps. As you can see in the output how Collaboratory will ask you to verify it by clicking in the given link then it will get mounted by giving a authentication to it.

```
with open ('/gdrive/My Drive/foo.txt', 'w') as f:
    f.write ('Hello Google Drive!')
!cat '/gdrive/My Drive/foo.txt'
```

Output:

Hello Google Drive!

This is the code which will help you to check whether your drive has been mounted with the Collaboratory or not.

After doing this we will access our dataset which has been uploaded in the drive directly and unzip it to access the files which we require in this project.

```
from zipfile import ZipFile
# specifying the zip file name
file_name = "plantvillage-dataset.zip" #path for location of
dataset
# opening the zip file in READ mode
with ZipFile (file_name, 'r') as zi:
# extracting all the files
print ('Extracting all the files now...')
zi.extractall ()
print ('Done!')
```

Output:

Extracting all the files now...

Done!

Here we are simply extracting all the data from the ZIP folder of PlantVillage for further use.

```
#splitting data into train and test
!pip install split-folders tqdm
```

Output:

Collecting split-folders

Downloading https://files.pythonhosted.org/packages/20/67/29dda743e6d23ac1ea3d16704d8bbb48d65faf3f1b1leaf53153b3da56c56/split_folders-0.3.1-py3-none-any.whl

Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (4.38.0)

Installing collected packages: split-folders

Successfully installed split-folders-0.3.1

```
!mkdir potato
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _
Early_blight' 'potato/early blight'
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _
Late_blight' 'potato/late blight'
! cp -r '/content/plantvillage dataset/segmented/Potato _ _ _
healthy' 'potato/healthy'
```

Here we are making the directories for potato disease detection into the folder which is been extracted from plant village dataset.

```
import split_folders
# Split with a ratio.
# To only split into training and validation set, set a tuple
to 'ratio', i.e, ' (.8, .2)'.
#split_folders.ratio ('color', output="dataset", seed=1337,
ratio= (.8, .1, .1)) # default values
split_folders.ratio ('potato', output="dataset", seed=1337,
ratio= (.8, 0.2)) # default values
```

Output:

Copying files: 2152 files [00:00, 9225.64 files/s]

Step 3: Building CNN model

First, we will build our CNN Model for which here is the code.

```
def model_cnn ():
    model = Sequential ([
        BatchNormalization (axis=1, input_shape= (256,256,3)),

        Convolution2D (32,3,3, activation='relu'),
        BatchNormalization (axis=1),
        MaxPooling2D ( (3,3)),
        Convolution2D (64,3,3, activation='relu'),
        BatchNormalization (axis=1),
        MaxPooling2D ((3,3)),
        Flatten (),
        Dense (200, activation='relu'),
        Dropout (0.2),
        BatchNormalization (),
        Dense (3, activation='softmax')
    ])
return model
```

The model type which we are using is the basic and easy to make any CNN model which is Sequential. Here we have defined whole model into a function so that we can call it whenever it is required.

And Sequential is the easiest method to build it by using Keras. It allows us to build the CNN model Layer by layer.

In the Model, our first Layer is the Convolution2D (32,3,3, Activation='relu'), It deals with the input images and the argument 32 Depicts the number of nodes in the layer which can be changed from time to time based on dataset size. Where the 3,3 in the argument depicts the kernel size which is the size of the filter matrix for our convolution layer in the model.so a kernel size of 3 means we have 3X3 filter matrix for our model layer. The Argument Activation is the activation function for the layer which is ReLU or Rectified Liner Activation which works good for the neural networks.

After that we will use Batch Normalization (axis=1) for normalizing and re scaling the data where argument axis=1 represents the channel. After which we will use max pooling 2D for selecting maximum features covered by the filters.

In the Next layer Convolution2D (64,3,3, Activation='relu') where In this layer we will have 64 nodes for the input images to process and continuously we will have both batch normalization and max pooling again.

In between the layers we have "flatten ()" layer which always server as the connection between the convolution and dense layer.

In the model we will have 2 output layers where first layer As a output layer we have Dense (200, activation='relu') layer. Dense is a standard type in neural network which can be used for many purposes. Here the argument 200 in dense layer means we will have 200 nodes in our output layer one for each possible outcome between 0-199. After this we are using dropout (0.2) which is used to drop the certain units from the CNN model which will help us to prevent over fitting in the model and an approach to regularization in the network which will help in reducing the interdependencies among the nodes of network. After this we will use batch Normalization in the layer for normalizing and re scaling the data. And the we have our final and 2nd output layer Dense (3, activation='softmax') which has 3 modes for every output from 0-2 and activation function as softmax.

We can also code the same model by using add function which is used to code layer by layer of CNN model.

Alternative code for above built model using add function:

Step 4: Now we will generate the data for model and then do the training.

```
train_datagen = ImageDataGenerator (rescale=1./255)
validation_datagen = ImageDataGenerator (rescale=1./255)
image_size = 256
# Change the batchsize according to your system RAM
train_batchsize = 20
val_batchsize = 20
train_generator = train_datagen.flow_from_directory (
    'dataset/train',
    target_size= (image_size, image_size),
    batch_size=train_batchsize,
    class_mode='categorical')

validation_generator = validation_datagen.flow_from_
    directory (
        'dataset/val',
        target_size= (image_size, image_size),
        batch_size=val_batchsize,
        class_mode='categorical',
        shuffle=True)
```

Output:

Found 1721 images belonging to 3 classes.

Found 431 images belonging to 3 classes

Here we are generating the dataset related to potato and dividing them into training and testing sets for our CNN model. As you can see in the code, we have declared image size and batch size which can be altered according to the need of a dataset and system.

Here we are making a function which has learning rate which will get change based on epoch changes so that we can get better result from the trained model.

Note: We can also keep a constant learning rate for all the epochs. It is totally our wish to do so. Sometimes if we don't get required accuracy while testing the model we do changes in the learning rate.

```
def step_decay (epoch):
if epoch<=6:
return 1e-3
elif epoch<=20:
return 1e-4
else:
return 1e-5

lr = LearningRateScheduler (step_decay)
```

Here will recall our model and start training the model by setting learning rate and optimizers values for better results which you can see in the outputs.

```
#define model
model = model_cnn ()

#calling the learning rate
callbacks = [lr]

# Compile the model
model.compile (loss='categorical_crossentropy',
optimizer=Adam (lr=0.0001, beta_1=0.9, beta_2=0.999,
epsilon=1e-08, decay=0.0), metrics=['acc'])
history = model.fit_generator (
train_generator,
steps_per_epoch=train_generator.samples/train_generator.
batch_size ,
validation_data=validation_generator,
validation_steps=validation_generator.samples/validation_
generator.batch_size,
epochs=23,
callbacks = callbacks)
```

Output:

```
Epoch 1/23
87/86 [=====] - 5s 54ms/step - loss: 0.5352 - acc:
0.7966 - val_loss: 0.7801 - val_acc: 0.6288 - lr: 0.0010
Epoch 2/23
87/86 [=====] - 5s 52ms/step - loss: 0.3022 - acc:
0.8879 - val_loss: 0.6320 - val_acc: 0.6914 - lr: 0.0010
Epoch 3/23
87/86 [=====] - 5s 53ms/step - loss: 0.2282 - acc:
0.9169 - val_loss: 0.3148 - val_acc: 0.9002 - lr: 0.0010
Epoch 4/23
87/86 [=====] - 5s 53ms/step - loss: 0.2011 - acc:
0.9303 - val_loss: 0.2739 - val_acc: 0.8585 - lr: 0.0010
Epoch 5/23
87/86 [=====] - 5s 52ms/step - loss: 0.1417 - acc:
0.9553 - val_loss: 0.6454 - val_acc: 0.7262 - lr: 0.0010
Epoch 6/23
87/86 [=====] - 5s 52ms/step - loss: 0.1223 - acc:
0.9599 - val_loss: 0.1182 - val_acc: 0.9397 - lr: 0.0010
Epoch 7/23
87/86 [=====] - 5s 53ms/step - loss: 0.0917 - acc:
0.9692 - val_loss: 0.2812 - val_acc: 0.8794 - lr: 0.0010
Epoch 8/23
87/86 [=====] - 5s 53ms/step - loss: 0.0757 - acc:
0.9773 - val_loss: 0.0892 - val_acc: 0.9722 - lr: 1.0000e-04
Epoch 9/23
87/86 [=====] - 5s 53ms/step - loss: 0.0691 - acc:
0.9791 - val_loss: 0.0852 - val_acc: 0.9606 - lr: 1.0000e-04
Epoch 10/23
87/86 [=====] - 5s 53ms/step - loss: 0.0526 - acc:
0.9878 - val_loss: 0.0890 - val_acc: 0.9722 - lr: 1.0000e-04
Epoch 11/23
87/86 [=====] - 5s 52ms/step - loss: 0.0864 - acc:
0.9843 - val_loss: 0.0945 - val_acc: 0.9675 - lr: 1.0000e-04
Epoch 12/23
87/86 [=====] - 5s 52ms/step - loss: 0.0527 - acc:
0.9872 - val_loss: 0.0767 - val_acc: 0.9698 - lr: 1.0000e-04
Epoch 13/23
87/86 [=====] - 4s 51ms/step - loss: 0.0470 - acc:
0.9872 - val_loss: 0.1524 - val_acc: 0.9420 - lr: 1.0000e-04
Epoch 14/23
87/86 [=====] - 4s 52ms/step - loss: 0.0486 - acc:
0.9901 - val_loss: 0.0918 - val_acc: 0.9652 - lr: 1.0000e-04
Epoch 15/23
87/86 [=====] - 5s 52ms/step - loss: 0.0417 - acc:
0.9895 - val_loss: 0.0852 - val_acc: 0.9652 - lr: 1.0000e-04
Epoch 16/23
87/86 [=====] - 4s 51ms/step - loss: 0.0420 - acc:
0.9942 - val_loss: 0.0744 - val_acc: 0.9768 - lr: 1.0000e-04
```



```

Epoch 17/23
87/86 [=====] - 4s 51ms/step - loss: 0.0424 - acc:
0.9919 - val_loss: 0.0953 - val_acc: 0.9675 - lr: 1.0000e-04
Epoch 18/23
87/86 [=====] - 4s 52ms/step - loss: 0.0372 - acc:
0.9895 - val_loss: 0.1163 - val_acc: 0.9606 - lr: 1.0000e-04
Epoch 19/23
87/86 [=====] - 5s 53ms/step - loss: 0.0315 - acc:
0.9965 - val_loss: 0.0718 - val_acc: 0.9675 - lr: 1.0000e-04
Epoch 20/23
87/86 [=====] - 4s 51ms/step - loss: 0.0371 - acc:
0.9919 - val_loss: 0.1132 - val_acc: 0.9698 - lr: 1.0000e-04
Epoch 21/23
87/86 [=====] - 4s 51ms/step - loss: 0.0275 - acc:
0.9948 - val_loss: 0.0712 - val_acc: 0.9722 - lr: 1.0000e-04
Epoch 22/23
87/86 [=====] - 4s 51ms/step - loss: 0.0345 - acc:
0.9930 - val_loss: 0.0813 - val_acc: 0.9768 - lr: 1.0000e-05
Epoch 23/23
87/86 [=====] - 4s 51ms/step - loss: 0.0306 - acc:
0.9942 - val_loss: 0.0875 - val_acc: 0.9768 - lr: 1.0000e-05

```

As you see that we have trained our model which gave us an accuracy of 99.42% and validation accuracy of 97.68%. After this we will plot our results using matplotlib.

Step 5: Summarizing the Model after training

```

# summarize history for accuracy
plt.plot (history.history['acc'])
plt.plot (history.history['val_acc'])
plt.title ('model accuracy')
plt.ylabel ('accuracy')
plt.xlabel ('epoch')
plt.legend (['train', 'test'], loc='upper left')
plt.show ()

```

Output: as in Fig. 10.1

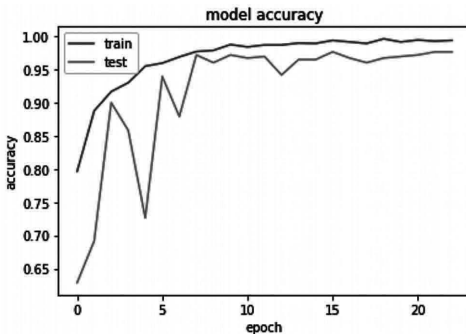


Fig. 10.1 Accuracy Representation of model

```
# summarize history for loss
plt.plot (history.history['loss'])
plt.plot (history.history['val_loss'])
plt.title ('model loss')
plt.ylabel ('loss')
plt.xlabel ('epoch')
plt.legend (['train', 'test'], loc='upper left')
plt.show ()
```

Output: as in Fig. 10.2

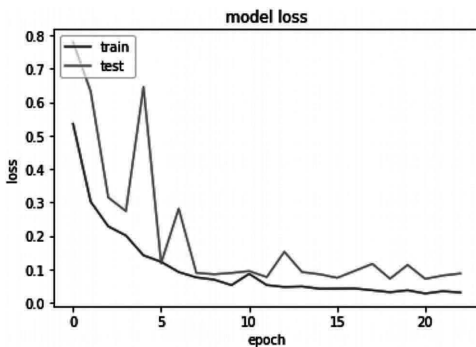


Fig. 10.2 Loss Representation of model

Here we have plotted our results between model accuracy and model loss. Which you can see in the graphs.

Step 6: Saving the model

```
model_json = model.to_json ()
with open ("model.json", "w") as json_file:
    json_file.write (model_json)
# serialize weights to HDF5
model.save_weights ("model.h5")
print ("Model Saved")
```

output:

Model Saved

In this step we are saving the trained model which can be utilized further. By doing this we no need to train our model again and again for any new data input to test their results.

Step 7: Testing the model

Here we will be testing the model which we have saved in earlier step-5.

```
#importing the model
```

```
from tensorflow.keras.models import model_from_json
json_file = open ('model.json', 'r')
loaded_model_json = json_file.read ()
json_file.close ()
loaded_model = model_from_json (loaded_model_json)
# load weights into new model
loaded_model.load_weights ('model.h5')
```

First we are loading our trained model which is ready to use for plant disease detection in potato and its healthy status.

```
class_dict = train_generator.class_indices
class_list = list (class_dict.keys ())
class_list
```

here we are creating a list for disease status of plant related to its categories

```
class_list=['early blight', 'healthy', 'late blight']
```

Here we are defining the list with disease names to predict while providing the input.

```
import cv2
def preprocess (image):
    image = cv2.cvtColor (image, cv2.COLOR_BGR2RGB)
    image = cv2.resize (image , (256,256))
    # create a simple mask image th the original one
    mask = np.zeros (image.shape[:2], np.uint8)

    # specify the background and foreground model
    backgroundModel = np.zeros ( (1, 65), np.float64)
    foregroundModel = np.zeros ( (1, 65), np.float64)

    # define the Region of Interest (ROI) as the coordinates of the
    rectangle
    rectangle = (20, 20, 220, 220)
    cv2.grabCut (image, mask,rectangle,
    backgroundModel, foregroundModel,
    1,cv2.GC_INIT_WITH_RECT)

    mask2 = np.where ( (mask == 2)| (mask == 0), 0, 1).astype ('uint8')
    image = image * mask2[:, :, np.newaxis]
    image = image/255
    return image
```

Here we have used Computer Vision for providing input to the trained model.

```
def predict_class (model,image_path):
```

```
original_image = cv2.imread (image_path)
image = preprocess (original_image)
tab = []
tab.append (image)
tab = np.array (tab)
h = np.expand_dims (tab, axis=2)
h=model.predict (tab)
predicted_class = np.argmax (h[0])
confidence = np.max (h[0])

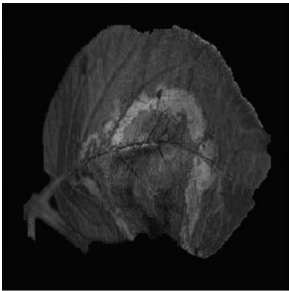
# plot results
plt.figure (figsize = (4,4))
plt.imshow (image)
plt.axis ('off')
plt.title ('prediction: '+class_list[predicted_class]+' with
conf '+str (confidence))
plt.show ()
```

here we are plotting the results for the input data images into the model.

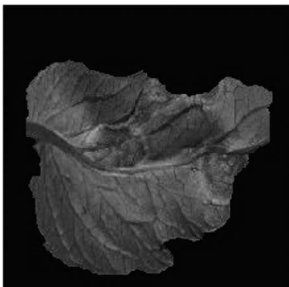
```
predict_class (loaded_model,'test/lateb.jpg')#image path is
provided for prediction
predict_class (loaded_model,'test/lateb2.jpg')
```

Output:

prediction late blight with conf 0.9999802



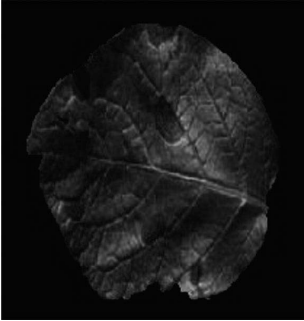
prediction late blight with conf 0.99996245



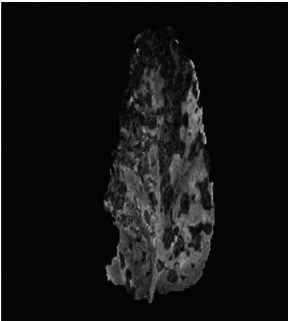
```
predict_class (loaded_model,'test/earlyb.jpg')  
predict_class (loaded_model,'test/earlybl.jpg')
```

Output:

prediction late blight with conf 0.9517815



prediction late blight with conf 9.5648595



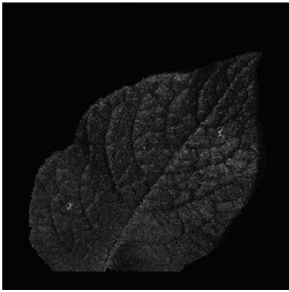
```
predict_class (loaded_model,'test/healthy.jpg')  
predict_class (loaded_model,'test/healthy1.jpg')
```

Output:

prediction late blight with conf 0.5035014



prediction late blight with conf 0.5035014



As you have in the project the how a CNN model is mad for plant disease detection and how we have saved the model for further future use. You can download the saved model from this link and can directly use it for prediction. # link is _____.

Summary

Plant disease classification has been a very prominent in agriculture which make it more useful for farmer and identify and to prevent it being spread among the plant so that damage to the crops can be prevented as in this chapter we have used plant village dataset to identify the disease in plant using the symptoms of it on plant leaves using machine learning, CNN and computer vision which are the domains of artificial Intelligence. The current project which have been made in this chapter can be used in real time for identifying diseases in potato plants. In the same way we can design an recognition and identification of diseases in plant can be designed for every plant which can benefit farmers in their farming process.